Initiation au Développement et à la Conception Logicielle

Thème principal : Conception, modélisation algorithmique et bonnes pratiques d'implémentation

Synthèse exécutive : Ce cours de la ressource S1.01 pose les fondements indispensables de la programmation et de la conception informatique en BUT 1 Informatique. Il formalise la démarche logique permettant de passer d'un problème métier réel à un algorithme structuré, puis à un code maintenable, testable et robuste.

Fondations de la modélisation et de l'analyse des besoins

Avant d'écrire la moindre ligne de code, un informaticien doit comprendre le problème à résoudre. La phase de spécification permet d'éviter les malentendus entre le client et le développeur. La décomposition modulaire garantit la lisibilité et la réutilisabilité du code produit.

- Spécification fonctionnelle : Description claire de ce que le programme doit accomplir du point de vue de l'utilisateur.
- Décomposition modulaire : Technique consistant à diviser un problème complexe en sous-problèmes plus simples et indépendants.

Cas concret enseignant : *Imaginons que vous devez développer un logiciel de caisse pour un commerce local. Si vous foncez coder sans réfléchir, vous oublierez la gestion de la monnaie rendue ou les réductions. Poser l'algorithme sur papier avec des modules clairs (calcul du total, application des remises, encaissement) évite les bugs majeurs.*

Bonnes pratiques d'implémentation et qualité logicielle

Un code qui fonctionne ne suffit pas. En BUT Informatique, l'accent est mis sur la propreté, le nommage explicite des variables et la documentation. Un code propre est un code qui s'explique de même, réduisant la dette technique.

- Maintenabilité du code : Facilité avec laquelle un code peut être modifié, corrigé ou étendu par un autre développeur.
- Jeu de tests unitaires : Ensemble de vérifications automatiques validant le comportement de chaque composant isolé.

Cas concret enseignant : *Le professeur insiste souvent sur le syndrome du 'code spaghetti' : un programme fonctionnel mais incompréhensible où tout est imbriqué. Si vous nommez vos variables 'x', 'y' et 'temp', personne ne comprendra votre code dans trois mois, pas même vous.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Est-ce qu'on est obligé de faire tout le diagramme et l'algorithme sur papier avant de coder sur l'IDE ?
Réponse du professeur : Oui, pour les projets de ce semestre, c'est obligatoire car cela force à structurer sa pensée et évite de se perdre dans le débogage syntaxique prématuré.

Question étudiant : Comment savoir si notre décomposition en modules est assez fine ?
Réponse du professeur : Un module doit avoir une seule et unique responsabilité claire. S'il fait plus de trente lignes ou traite de plusieurs sujets différents, il faut le redécouper.

■ Exercice & Démonstration d'Application

Énoncé : Écrire l'algorithme d'une fonction qui prend en paramètre un tableau d'entiers et retourne la valeur maximale contenue dans ce tableau.

Démarche de résolution :
1. Initialiser une variable 'max' avec le premier élément du tableau (indice 0). 2. Parcourir le tableau du second

élément (indice 1) jusqu'au dernier. 3. Pour chaque élément, comparer sa valeur avec la variable 'max'. 4. Si l'élément courant est strictement supérieur à 'max', alors affecter la valeur de l'élément courant à 'max'. 5. À la fin du parcours, retourner la variable 'max'.

```
/**
 * Fichier : max_tableau.cpp
 * Auteur : Matéi de Saint Leger
 * Rôle : Recherche de la valeur maximale dans un tableau d'entiers
 */

#include <iostream>
#include <vector>

// CONSTANTES

// VARIABLES
// (clavier) >> saisir >> taille_tableau
// (memoire) >> allouer >> tableau T

int trouveMax(const std::vector<int>& T)
{
    // VARIABLES LOCALES DE LA FONCTION
    int max;

    // TRAITEMENTS
    max = T[0];
    for (size_t i = 1; i < T.size(); ++i)
    {
        if (T[i] > max)
        {
            max = T[i];
        }
    }
    return max;
}

int main()
{
    std::vector<int> T = {5, 12, 3, 9, 21, 8};
    int resultat = trouveMax(T);
    std::cout << "Le maximum est : " << resultat << std::endl;
    return 0;
}
```

Résultat : Fonction trouveMax(tableau T) : retourne un entier représentant la plus grande valeur du tableau en complexité linéaire O(n).

■ Auto-Évaluation Flash d'Examen

Q : Quel est l'objectif principal de la décomposition modulaire ?

R : Diviser un problème complexe en sous-problèmes plus simples, indépendants, lisibles et réutilisables.

Q : Pourquoi est-il crucial de rédiger un jeu de tests ?

R : Pour valider automatiquement le comportement de chaque composant et s'assurer qu'aucune régression n'est introduite lors des modifications.

Q : Qu'appelle-t-on la maintenabilité d'un logiciel ?

R : La capacité d'un code à être facilement compris, corrigé, modifié ou étendu par un autre développeur.